

Chapters 6, 7: Contrast coding

Shravan Vasishth

2026-03-27

Contents

1	Introduction	2
2	Preliminaries: Matrix notation of the linear model and the generalized matrix inverse	4
2.1	Example: The Persian 2×2 design	7
3	Contrast coding	10
3.1	Basic concepts illustrated using a two-level factor	10
3.2	Default contrast coding: Treatment contrasts	11
3.3	Defining comparisons	13
3.4	Sum contrasts	15
3.5	The hypothesis matrix illustrated with a three-level factor	18
3.6	Sum contrasts	20
3.7	The hypothesis matrix	21
3.8	Generating contrasts: The <i>hypr</i> package	26
3.9	Example 1: Four conditions, treatment contrasts	29
3.10	Example 2: Four conditions, Helmert contrasts	31
3.11	Example 3: Four conditions, sliding or repeated differences or successive differences contrast	33
4	Contrast coding for multifactorial designs	35
4.1	ANOVA-style coding	35
4.2	Practical relevance of scaling for prior specification for interactions	43
4.3	Using <i>ifelse</i> to code contrasts	44
4.4	Using <i>merge</i> to code contrasts	45
4.5	Nested effects	46

Please also read chapters 6 and 7 of the textbook.

1 Introduction

The traditional approach to data analysis in psychology/psycholinguistics proceeds as follows.

People generally first do an omnibus ANOVA to check if there is **any** difference at all between, say, three conditions (regardless of what those differences might be). Here, I use an example that will turn up below as well: we have three levels to a factor, Nouns, Verbs, and Adjectives, and we are investigating relative reading times in these conditions. This is simulated data.

```
data("df_contrasts2")
head(df_contrasts2)

## # A tibble: 6 x 3
##   F      DV   id
##   <fct> <int> <int>
## 1 nouns  476    1
## 2 nouns  517    2
## 3 nouns  491    3
## 4 nouns  516    4
## 5 verbs  464    5
## 6 verbs  439    6

## load library implementing anova:
library(rstatix)
omnibus_anova<-anova_test(data = df_contrasts2,
                          dv = DV,
                          between = F
)

get_anova_table(omnibus_anova)

## ANOVA Table (type II tests)
##
##   Effect DFn DFd      F      p p<.05 ges
## 1      F    2    9 24.962 0.000213 * 0.847
```

The conventional procedure is to check whether there is an overall significant effect, then and only then do they do so-called post-hoc t-tests, pairwise.

In the above omnibus anova, we see a significant difference between the conditions, but we can't know which conditions differ from which. So we proceed as follows.

Get the data for each condition:

```
N<-subset(df_contrasts2,F=="nouns")$DV
V<-subset(df_contrasts2,F=="verbs")$DV
A<-subset(df_contrasts2,F=="adjectives")$DV
```

Then do pairwise comparisons; for example:

```
t.test(N,A)

##
##  Welch Two Sample t-test
##
## data:  N and A
## t = 7.0729, df = 5.9997, p-value = 0.0004004
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##   65.24066 134.25934
## sample estimates:
## mean of x mean of y
##   500.00   400.25

t.test(V,A)

##
##  Welch Two Sample t-test
##
## data:  V and A
## t = 3.5463, df = 5.9997, p-value = 0.01213
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##   15.49976 84.50024
## sample estimates:
## mean of x mean of y
##   450.25   400.25
```

This conventional method is pointless: in reality, the researcher is usually interested in specific pairwise comparisons; we might as well test those directly. Knowledge of contrast coding makes this direct estimation/hypothesis testing possible.

Before we get into contrast coding, a reminder about the matrix formulation of the linear model is necessary.

2 Preliminaries: Matrix notation of the linear model and the generalized matrix inverse

The question we address here is: given the data y and some predictor(s) x , how to estimate the parameters β ?

Consider a *deterministic* function $\phi(\mathbf{x}, \beta)$ which takes as input some variable values x and some fixed values β . A simple example would be

$$y = \beta x \quad (1)$$

Another example with two fixed values β_0 and β_1 is:

$$y = \beta_0 + \beta_1 x \quad (2)$$

We can rewrite the above equation as follows.

$$\begin{aligned} y &= \beta_0 + \beta_1 x \\ &= \beta_0 \times 1 + \beta_1 x \\ &= \begin{pmatrix} 1 & x \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix} \\ y &= \phi(x, \beta) \end{aligned} \quad (3)$$

In a statistical model, we don't expect an equation like $y = \phi(x, \beta)$ to fit all the points exactly. For example, we could come up with an equation that, given a person's weight, we can compute their height:

$$\text{height} = \beta_0 + \beta_1 \text{weight} \quad (4)$$

Given any single value of the weight of a person, we will probably not get a perfectly correct prediction of the

height of that person. This leads us to a *non-deterministic* version of the above function:

$$y = \phi(x, \beta, \varepsilon) = \beta_0 + \beta_1 x + \varepsilon \quad (5)$$

Here, ε is an error random variable which is assumed to have some probability density function (specifically, the normal distribution) associated with it. It is assumed to have expectation (mean) 0, and some standard deviation (to be estimated from the data) σ . We can write this statement in compact form as $\varepsilon \sim N(0, \sigma)$.

The **general linear model** is a *non-deterministic* function like the one above (the superscript T is the transformation operation on a matrix, which converts the rows of a matrix into the columns, and vice versa):

$$Y = f(x)^T \beta + \varepsilon \quad (6)$$

The matrix formulation will be written as below. n refers to the number of data points (that is, $Y = y_1, \dots, y_n$), and the index i ranges from 1 to n .

$$Y = X\beta + \varepsilon \Leftrightarrow y_i = f(x_i)^T \beta + \varepsilon_i, i = 1, \dots, n \quad (7)$$

To make this concrete, suppose we have three data points, i.e., $n = 3$. Then, the matrix formulation is

$$\begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & x_3 \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix} + \varepsilon \quad (8)$$

$$Y = X\beta + \varepsilon$$

Here, $f(x_1)^T = (1 \ x_1)$, and is the first row of the matrix X , $f(x_2)^T = (1 \ x_2)$ is the second row, and $f(x_3)^T = (1 \ x_3)$ is the third row.

Note that the expectation of Y , $E[Y]$, is $X\beta$. β is a $p \times 1$ matrix, and X , the **design matrix**, has dimensions $n \times p$.

We now provide a geometric argument for least squares estimation of the β parameters. When we have a *deterministic* model $y = \phi(f(x)^T, \beta) = \beta_0 + \beta_1 x$, this implies a perfect fit to all data points.

This is like solving the equation $Ax = b$ in linear algebra: we solve for β in $X\beta = y$ using, e.g., Gaussian elimination.

But when we have a *non-deterministic* model $y = \phi(f(x)^T, \beta, \varepsilon)$, there is no solution. Now, the best we can do is to get Ax to be as close an approximation as possible to b in $Ax = b$. In other words, we try to minimize $|b - Ax|$.

The goal is to estimate β ; we want to find a value of β such that the observed Y is as close to its expected value $X\beta$.

In order to be able to identify β from $X\beta$, the linear transformation $\beta \rightarrow X\beta$ should be one-to-one, so that every possible value of β gives a different $X\beta$. This in turn requires that X be of full rank p . (Rank refers to the number of linearly independent columns or rows. The row rank and column rank of an $m \times n$ matrix will be the same, so we can just talk of rank of a matrix. An $m \times n$ matrix X with $\text{rank}(X) = \min(m, n)$ is called full rank.)

So, if X is an $n \times p$ matrix, then it is necessary that $n \geq p$. There must be at least as many observations as parameters. If this is not true, i.e., if there are more parameters than data-points, then the model is said to be **over-parameterized**.

Assuming that X is of full rank, and that $n > p$, Y can be considered a point in n -dimensional space and the set of candidate $X\beta$ is a p -dimensional subspace of this space; see Figure 1.

There will be one point in this subspace which is closest to Y in terms of Euclidean distance. The unique β that corresponds to this point is the **least squares estimator** of β ; we will call this estimator $\hat{\beta}$.

Notice that $\varepsilon = (Y - X\hat{\beta})$ and $X\hat{\beta}$ are perpendicular to each other. Because the dot product of two perpendicular (orthogonal) vectors is 0, we get the result:

$$(Y - X\hat{\beta})^T X\hat{\beta} = 0 \Leftrightarrow (Y - X\hat{\beta})^T X = 0 \quad (9)$$

Multiplying out the terms, we proceed as follows. One result that we use here is that $(AB)^T = B^T A^T$.

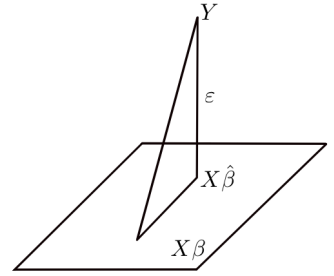


Figure 1: Geometric interpretation of least squares.

$$\begin{aligned}
(Y - X\hat{\beta})^T X &= 0 \\
(Y^T - \hat{\beta}^T X^T) X &= 0 \\
\Leftrightarrow Y^T X - \hat{\beta}^T X^T X &= 0 \\
\Leftrightarrow Y^T X &= \hat{\beta}^T X^T X \\
\Leftrightarrow (Y^T X)^T &= (\hat{\beta}^T X^T X)^T \\
\Leftrightarrow X^T Y &= X^T X \hat{\beta}
\end{aligned} \tag{10}$$

This gives us the important result:

$$\hat{\beta} = (X^T X)^{-1} X^T Y \tag{11}$$

This is a key and famous result for linear models. Here, the transformation of the design matrix X into $(X^T X)^{-1} X^T$ plays a central role for estimating regression coefficients. Indeed, this term $(X^T X)^{-1} X^T$ is exactly the **generalized matrix inverse** of the design matrix X , which we can compute in R using the command `MASS::ginv()` or `matlib::Ginv()`.

2.1 Example: The Persian 2×2 design

```
library(bcogsci)
data("df_persianE1")
dat<-df_persianE1
head(dat)

##      subj item   rt distance  predability
## 60      4    6  568    short predictable
## 94      4   17  517     long unpredictable
## 146     4   22  675    short predictable
## 185     4    5  575     long unpredictable
## 215     4    3  581     long predictable
## 285     4    7 1171     long predictable

## create a condition column for convenience,
## to map the condition levels to the above table:
dat$condition<-ifelse(dat$distance=="long" &
                      dat$predability=="unpredictable",
                      "a",
```

```

    ifelse(dat$distance=="short" & dat$predability=="unpredictable","b",
           ifelse(dat$distance=="long" & dat$predability=="predictable",
                  "c","d")))

dat$pred<-ifelse(dat$condition%in%c("a","b"),0.5,-0.5)
dat$dist<-ifelse(dat$condition%in%c("a","c"),0.5,-0.5)
dat$int<-dat$pred*dat$dist

## this model is wrong for these data, but just
## illustrating the matrix computations:
m<-lm(rt~pred+dist+int,dat)
coef(m)

## (Intercept)      pred      dist      int
##  583.69444    54.03704    55.81746    27.85185

## by hand:
X<-model.matrix(m)
y<-dat$rt
## X+:
X_plus<-solve(t(X)%*%X)%*%t(X)
dim(X_plus)

## [1]      4 1512

## solve is the function that computes the inverse:
beta_hat<-X_plus%*%y

## compare:
coef(m)

## (Intercept)      pred      dist      int
##  583.69444    54.03704    55.81746    27.85185

t(beta_hat)

##      (Intercept)      pred      dist      int
## [1,]  583.6944  54.03704  55.81746  27.85185

```

Conceptually, the contrast matrix can be represented in two ways:

- one representation defines the **design or model matrix** X used to define independent variables in the linear regression model $y = X\beta$
- the other representation, the generalized matrix inverse of the design matrix $X^+ = (X^T X)^{-1} X^T$, defines **weights for how observed data are combined to obtain estimated regression coefficients** via $\hat{\beta} = X^+ y$ and for what formal hypotheses are tested via the contrasts.

Given the important property of the generalized matrix inverse that applying it twice yields back the original matrix,

```
head(MASS::ginv(X_plus))
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1 -0.5 -0.5  0.25
## [2,]    1  0.5  0.5  0.25
## [3,]    1 -0.5 -0.5  0.25
## [4,]    1  0.5  0.5  0.25
## [5,]    1 -0.5  0.5 -0.25
## [6,]    1 -0.5  0.5 -0.25
```

the generalized matrix inversion operation can be used to flip back and forth between these two representations of the design matrix.

The generalized inverse of the design matrix, X^+ , defines weights for each of the k factor levels of the independent variables.

As estimating coefficients for all k factor levels in addition to the intercept is redundant, the design matrix X defines coefficients only for $k - 1$ comparisons. The generalized matrix inversion can be seen as transforming between these two spaces. A vignette explaining the generalized inverse is available for the `Ginv()` function in the `matlib` package.

3 Contrast coding

3.1 Basic concepts illustrated using a two-level factor

We first consider the simplest case: suppose we want to compare the means of a dependent variable (DV) such as response times between two groups of subjects. A simulated data set is available in the package `bcogsci` as the data set `df_contrasts1`. The simulations assumed longer response times in condition $F1$ ($\mu_1 = 0.8$ sec) than $F2$ ($\mu_2 = 0.4$ sec). The data from the 10 simulated subjects are aggregated and summary statistics are computed for the two groups.

```
data("df_contrasts1")
df_contrasts1
```

```
## # A tibble: 10 x 3
##   F      DV    id
##   <fct> <dbl> <int>
## 1 F1    0.636    1
## 2 F1    0.841    2
## 3 F1    0.555    3
## 4 F1    1.03     4
## 5 F1    0.938    5
## 6 F2    0.123    6
## 7 F2    0.304    7
## 8 F2    0.659    8
## 9 F2    0.469    9
## 10 F2    0.444   10

## [1] 0.6
```

```
Factor N data Est. means Std. dev. Std. errors 1 F1 5 0.8
0.2 0.1 2 F2 5 0.4 0.2 0.1
```

The results, displayed in Figure 2 and in the Table above, show that the assumed true condition means are exactly realized with the simulated data. The numbers are exact because the `mvrnorm()` function used here (see `?df_contrasts1`) ensures that the data are generated so that the sample mean yields the true means for each level.

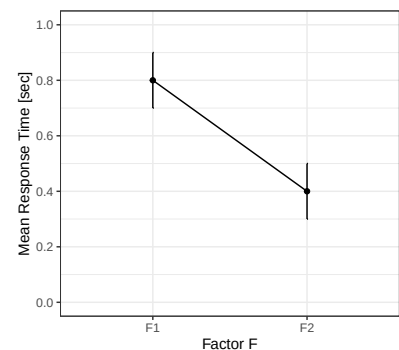


Figure 2: Means and standard errors of the simulated dependent variable (e.g., response times in seconds) in two conditions $F1$ and $F2$

In real data sets, of course, the sample means will vary from experiment to experiment.

A simple Bayesian linear model of DV on F yields a straightforward estimate of the difference between the group means. We use relatively uninformative priors. The estimates for the population-level effects are presented below using the function `fixef()`:

```
fit_F <- brm(DV ~ 1 + F,
  data = df_contrasts1,
  family = gaussian(),
  prior = c(
    prior(normal(0, 2), class = Intercept),
    prior(normal(0, 2), class = sigma),
    prior(normal(0, 1), class = b)
  )
)

fixef(fit_F)
```

##	Estimate	Est.Error	Q2.5	Q97.5
## Intercept	0.7948102	0.1100417	0.5579436	1.0107002
## FF2	-0.3921554	0.1576367	-0.6951845	-0.0614466

Comparing the means for each condition with the coefficients (Estimates) reveals that (i) the intercept (0.8) is the mean for condition $F1$, $\hat{\mu}_1$; and (ii) the slope (FF2: -0.4) is the difference between the estimated means for the two groups, $\hat{\mu}_2 - \hat{\mu}_1$:

$$\begin{aligned} \text{Intercept} &= \hat{\mu}_1 &&= \text{estimated mean for } F1 \\ \text{Slope (FF2)} &= \hat{\mu}_2 - \hat{\mu}_1 &&= \text{estim. mean for } F2 - \text{estim. mean for } F1 \end{aligned} \quad (12)$$

The new information is the 95% credible interval for the difference between the two groups.

3.2 Default contrast coding: Treatment contrasts

How does the function `brm` arrive at these particular values for the intercept and slope? That is, why does the intercept assess the mean of condition $F1$ and how do we

know the slope measures the difference in means between $F2-F1$?

This result is a consequence of the default contrast coding of the factor F . R assigns treatment contrasts to factors and orders their levels alphabetically.

The alphabetically first factor level (here: $F1$) is coded in R by default as 0 and the second level (here: $F2$) is coded as 1.¹ This becomes clear when we inspect the current contrast attribute of the factor using the `contrasts()` command:

```
contrasts(df_contrasts1$F)

##      F2
## F1    0
## F2    1
```

¹ When using treatment coding, it's important to be aware that the intercept won't be centered. This means the prior one sets for the intercept in brms will actually be for assigned to its centered version. Although this distinction may not have any impact when one uses wide priors, it's still worth keeping it in mind.

Why does this contrast coding yield these particular regression coefficients? Let's take a look at the regression equation.

Let α represent the intercept, and β_1 the slope. Then, the simple regression above expresses the belief that the expected response time $E[Y]$ is a linear function of the factor F .

$$E[Y] = \alpha + \beta_1 x \quad (13)$$

So, if $x = 0$ (condition $F1$), the expectation is $\alpha + \beta_1 \cdot 0 = \alpha$; and if $x = 1$ (condition $F2$), the expectation is $\alpha + \beta_1 \cdot 1 = \alpha + \beta_1$.

Expressing the above in terms of the estimated coefficients:

$$\begin{aligned} \text{estim. value for } F1 &= \hat{\mu}_1 = \hat{\alpha} = \text{Intercept} \\ \text{estim. value for } F2 &= \hat{\mu}_2 = \hat{\alpha} + \hat{\beta}_1 = \text{Intercept} + \text{Slope (FF2)} \end{aligned} \quad (14)$$

It is useful to think of such unstandardized regression coefficients as difference scores; they express the increase in the dependent variable y associated with a change in the independent variable x of 1 unit, such as going from

0 to 1 in this example. The difference between condition means is $0.4 - 0.8 = -0.4$, which is the estimated regression coefficient $\hat{\beta}_1$. The sign of the slope is negative because we have chosen to subtract the larger mean $F1$ score from the smaller mean $F2$ score.

3.3 Defining comparisons

The analysis of the regression equation demonstrates that in the treatment contrast the intercept assesses the average response in the baseline condition, whereas the slope estimates the difference between condition means. However, these are just verbal descriptions of what each coefficient assesses. Is it also possible to formally write down what each coefficient assesses?

From the perspective of parameter estimation, the slope represents the effect of main interest, so we consider this first.

The treatment contrast specifies that the slope β_1 estimates the difference in means between the two levels of the factor F . This can formally be written as:

$$\beta_1 = \mu_{F2} - \mu_{F1} \quad (15)$$

or equivalently:

$$\beta_1 = -1 \cdot \mu_{F1} + 1 \cdot \mu_{F2} \quad (16)$$

The ± 1 weights in the parameter estimation directly express which means are compared by the treatment contrast.

The intercept in the treatment contrast estimates a quantity that is usually of little interest: it estimates the mean in condition $F1$. Formally, the parameter α estimates the following quantity:

$$\alpha = \mu_{F1} \quad (17)$$

or equivalently:

$$\alpha = 1 \cdot \mu_{F1} + 0 \cdot \mu_{F2}. \quad (18)$$

The fact that the intercept term formally estimates the mean of condition *F1* is in line with our previous derivation (see equation 12).

In R, factor levels are ordered alphabetically and by default the first level is used as the baseline in treatment contrasts. Obviously, this default mapping will depend on the levels' alphabetical ordering.

If a different baseline condition is desired, it is possible to re-order the levels. Here is one way of re-ordering the levels:

```
df_contrasts1$Fb <- factor(df_contrasts1$F,
  levels = c("F2", "F1")
)
contrasts(df_contrasts1$Fb)

##      F1
## F2    0
## F1    1
```

This re-ordering did not change any data associated with the factor, only one of its attributes. With this new contrast attribute, a simple Bayesian model yields the following result.

```
fit_Fb <- brm(DV ~ 1 + Fb,
  data = df_contrasts1,
  family = gaussian(),
  prior = c(
    prior(normal(0, 2), class = Intercept),
    prior(normal(0, 2), class = sigma),
    prior(normal(0, 1), class = b)
  )
)

fixef(fit_Fb)

##              Estimate Est.Error      Q2.5      Q97.5
## Intercept 0.4054538 0.1117579 0.17268966 0.6325923
## FbF1       0.3913747 0.1553022 0.08828023 0.7136149
```

The model now estimates different quantities. The intercept now codes the mean of condition *F2*, and the

slope measures the difference in means between $F1$ minus $F2$. This represents an alternative coding of the treatment contrast.

These model posteriors do not provide evidence for the hypothesis that the effect of factor F is different from zero. If the research focus is on such hypothesis testing, Bayesian hypothesis tests can be carried out using Bayes factors, by comparing a model containing a contrast of interest with a model lacking this contrast. Bayes factors are discussed in another lecture (relating to chapter 13 of the book).

3.4 Sum contrasts

Treatment contrasts are only one of many options. It is also possible to use sum contrasts, which code one of the conditions as -1 and the other as $+1$, effectively *centering* the effects at the grand mean (GM, i.e., the mean of the two group means). Here, we rescale the contrast to values of -0.5 and $+0.5$, which makes the estimated treatment effect the same as for treatment coding and easier to interpret.

To define this contrast in a linear regression, one way is to use the `contrasts` function (another way is to define a column containing $+0.5$ and -0.5 for the corresponding levels of the factor).

```
contrasts(df_contrasts1$F) <- c(-0.5, +0.5)
fit_mSum <- brm(DV ~ 1 + F,
  data = df_contrasts1,
  family = gaussian(),
  prior = c(
    prior(normal(0, 2), class = Intercept),
    prior(normal(0, 2), class = sigma),
    prior(normal(0, 1), class = b)
  )
)
fixef(fit_mSum)
```

##	Estimate	Est.Error	Q2.5	Q97.5
----	----------	-----------	------	-------

```
## Intercept 0.5990816 0.07800925 0.4406240 0.7596245
## F1        -0.3936482 0.15040498 -0.6937032 -0.1015124
```

Here, the slope ($F1$) again codes the difference of the groups associated with the first and second factor levels. It has the same value as in the treatment contrast. One important difference from the treatment contrast is that the intercept now represents the estimate of the average of condition means for $F1$ and $F2$, that is, the grand mean. For the scaled sum contrast:

$$\begin{aligned}\text{Intercept} &= (\hat{\mu}_1 + \hat{\mu}_2)/2 = \text{estimated mean of } F1 \text{ and } F2 \\ \text{Slope (F1)} &= \hat{\mu}_2 - \hat{\mu}_1 = \text{estim. mean of } F2 - \text{estim. mean for } F1\end{aligned}\tag{19}$$

Why does the intercept assess the grand mean and why does the slope estimate the group difference? This is the result of rescaling the sum contrast. The first factor level ($F1$) was coded as -0.5 , and the second factor level ($F2$) as $+0.5$:

```
contrasts(df_contrasts1$F)

##      [,1]
## F1 -0.5
## F2  0.5
```

Look again at the regression equation to better understand what computations are performed. Again, α represents the intercept, β_1 represents the slope, and the predictor variable x represents the factor F . The regression equation is written as:

$$E[Y] = \alpha + \beta_1 x \tag{20}$$

The group of $F1$ subjects is then coded as -0.5 , and the response time for the group of $F1$ subjects is $\alpha + \beta_1 \cdot x_1 = 0.6 + (-0.4) \cdot (-0.5) = 0.8$. By contrast, the $F2$ group is coded as $+0.5$. By implication, the mean of the $F2$ group must be $\alpha + \beta_1 \cdot x_1 = 0.6 + (-0.4) \cdot 0.5 = 0.4$. Expressed in terms of the estimated coefficients:

$$\begin{aligned}
\text{estim. value for } F1 &= \hat{\mu}_1 = \hat{\alpha} - 0.5 \cdot \hat{\beta}_1 = \text{Intercept} - 0.5 \cdot \text{Slope (F1)} \\
\text{estim. value for } F2 &= \hat{\mu}_2 = \hat{\alpha} + 0.5 \cdot \hat{\beta}_1 = \text{Intercept} + 0.5 \cdot \text{Slope (F1)}
\end{aligned}
\tag{21}$$

The unstandardized regression coefficient is a *difference score*: Taking a step of one unit on the predictor variable x , e.g., from -0.5 to $+0.5$, reflecting a step from condition $F1$ to $F2$, changes the dependent variable from 0.8 (for condition $F1$) to 0.4 (condition $F2$). This reflects a difference of $0.4 - 0.8 = -0.4$; this is again the estimated regression coefficient $\hat{\beta}_1$.

Moreover, as mentioned above, the intercept now assesses the grand mean, i.e., the unweighted mean of the means for conditions $F1$ and $F2$: it is in the middle between condition means for $F1$ and $F2$.

So far we gave verbal statements about what is estimated by the intercept and the slope in the case of the scaled sum contrast. It is possible to write these statements as formal parameter estimates. In sum contrasts, the slope parameter β_1 assesses the following quantity:

$$\beta_1 = -1 \cdot \mu_{F1} + 1 \cdot \mu_{F2} \tag{22}$$

This estimates the same quantity as the slope in the treatment contrast. The intercept now assesses a different quantity: the average of the two conditions $F1$ and $F2$:

$$\alpha = 1/2 \cdot \mu_{F1} + 1/2 \cdot \mu_{F2} = \frac{\mu_{F1} + \mu_{F2}}{2} \tag{23}$$

In balanced data, i.e., in data sets where there are no missing data points, the average of the two conditions $F1$ and $F2$ is the grand mean. In unbalanced data sets, where there are missing values, this average is the weighted grand mean.

To illustrate this point, consider an example with fully balanced data and two equal group sizes of 5 subjects for each group $F1$ and $F2$. Here, the grand mean is also the mean across all subjects. Next, consider a highly simplified unbalanced data set, where in condition $F1$ two observations of the dependent variable are available with

values of 2 and 3, and where in condition $F2$ only one observation of the dependent variable is available with a value of 4. In this data set, the mean across all subjects is $\frac{2+3+4}{3} = \frac{9}{3} = 3$. However, the (weighted) grand mean as assessed in the intercept in a model using sum contrasts for factor F would first compute the mean for each group separately (i.e., $\frac{2+3}{2} = 2.5$, and 4), and then compute the mean across conditions $\frac{2.5+4}{2} = \frac{6.5}{2} = 3.25$. The grand mean of 3.25 is different from the mean across subjects of 3.

To summarize, **treatment contrasts** and **sum contrasts** are two possible ways to parameterize the difference between two groups; they generally estimate different quantities.

Treatment contrasts compare one or more means against a baseline condition, whereas sum contrasts compare a condition's mean to the grand mean (which in the two-group case also implies estimating the difference between the two group means).

One question that comes up here is: how does one know or how can one formally derive what quantities are estimated by a given set of contrasts? (In the context of Bayes factors, the question would be: what hypothesis test does the contrast coding encode?) This question will be discussed in detail below for the general case of any arbitrary contrast coding.

3.5 *The hypothesis matrix illustrated with a three-level factor*

Consider an example with the three word classes nouns, verbs, and adjectives. We load simulated data from a lexical decision task with response times as dependent variable. The research question is: do response times differ as a function of the between-subject factor word class with three levels: nouns, verbs, and adjectives? Here, just to illustrate the case of a three-level factor, we make the arbitrary assumption that nouns have longest response times and adjectives the shortest response times. Word class is specified as a between-subject factor. In cogni-

tive science experiments, word class will usually vary within subjects and between items. Because the within- or between-subjects status of an effect is independent of its contrast coding, we assume the manipulation to be between subjects for ease of exposition. The concepts presented here extend to repeated measures designs that are often analyzed using hierarchical Bayesian (linear mixed) models. (In these cases, there is the additional difficulty that for factors with many levels, the covariance matrix of the random effects can easily become very large.)

Load and display the simulated data:

```
data("df_contrasts2")
head(df_contrasts2)
```

```
## # A tibble: 6 x 3
##   F      DV   id
##   <fct> <int> <int>
## 1 nouns   476    1
## 2 nouns   517    2
## 3 nouns   491    3
## 4 nouns   516    4
## 5 verbs   464    5
## 6 verbs   439    6
```

```
table.word1
```

##	Factor	N data	Est. means	Std. dev.	Std. errors
## 1	adjectives	4	400.25	19.87251	9.936255
## 2	nouns	4	500.00	20.01666	10.008330
## 3	verbs	4	450.25	20.00625	10.003125

As shown in the output above, the estimated means reflect our assumptions about the true means in the data simulation: Response times are longest for nouns and shortest for adjectives.

In the following sections, we use this data set to illustrate sum contrasts. Furthermore, we will use an additional data set to illustrate : custom contrasts.

In practice, usually only one set of contrasts is selected when the expected pattern of means is formulated during

the design of the experiment. However, nothing prevents us from fitting more than one model with different contrast codings.

3.6 Sum contrasts

We begin with sum contrasts. Suppose that the expectation is that nouns are responded to slower and adjectives are responded to faster than the grand mean response time. Then, the research question could be: By how much do nouns differ from the grand mean and by how much do adjectives differ from the grand mean? And are the responses slower or faster than the grand mean? We want to estimate the following two quantities:

$$\beta_1 = \mu_1 - \frac{\mu_1 + \mu_2 + \mu_3}{3} = \mu_1 - GM \quad (24)$$

and

$$\beta_2 = \mu_2 - \frac{\mu_1 + \mu_2 + \mu_3}{3} = \mu_2 - GM \quad (25)$$

β_1 can also be written as:

$$\beta_1 = \mu_1 - \frac{\mu_1 + \mu_2 + \mu_3}{3} \quad (26)$$

$$\Leftrightarrow \beta_1 = \frac{2}{3}\mu_1 - \frac{1}{3}\mu_2 - \frac{1}{3}\mu_3 \quad (27)$$

Here, the weights $2/3, -1/3, -1/3$ are informative about how to combine the condition means to estimate the linear model coefficient.

β_2 is also rewritten as:

$$\beta_2 = \mu_2 - \frac{\mu_1 + \mu_2 + \mu_3}{3} \quad (28)$$

$$\Leftrightarrow \beta_2 = -\frac{1}{3}\mu_1 + \frac{2}{3}\mu_2 - \frac{1}{3}\mu_3 \quad (29)$$

Here, the weights are $-1/3, 2/3, -1/3$, and they again indicate how to combine the condition means for estimating the regression coefficient.

3.7 *The hypothesis matrix*

The weights of the condition means are not only useful for defining parameter estimates. They also provide the starting step in a very powerful method which allows the researcher to generate the contrasts that are needed to estimate these comparisons in a linear model. That is, what we did so far is to explain some kinds of different contrast codings that exist and what comparisons are estimated by these contrasts. That is, if a certain data set is given and the goal is to estimate certain comparisons, then the procedure would be to check whether any of the contrasts that we encountered estimate these comparisons of interest. Sometimes it suffices to use one of these existing contrasts. At other times, our research questions may not correspond exactly to any of the contrasts in the default set of standard contrasts provided in R. For these cases, or for more complex designs, it is very useful to know how contrast matrices are created. Indeed, a relatively simple procedure exists in which we

- write our comparisons formally,
- extract the weights of the condition means from the comparisons, and then
- automatically generate the correct **contrast matrix** that we need in order to estimate these comparisons in a linear model.

Using this powerful method, it is not necessary to find a match to a contrast matrix provided by the family of functions in R starting with the prefix `contr`.

Instead, it is possible to simply define the comparisons that one wants to estimate, and to obtain the correct contrast matrix for these in an automatic procedure.

Here, for pedagogical reasons, we show some examples of how to apply this procedure in cases where the comparisons *do* correspond to some of the existing contrasts.

Defining a custom contrast matrix involves four steps:

1. Write down the estimated comparisons

2. Extract the **weights** and write them into what we will call a *hypothesis matrix* (and can also be viewed as a *comparison matrix*)
3. Apply the *generalized matrix inverse* to the hypothesis matrix to create the contrast matrix
4. Assign the contrast matrix to the factor and run the (Bayesian) model

The term **hypothesis matrix** is used here because contrast coding is often done to carry out an explicit hypothesis test; but one could of course use contrast coding just to compute the **estimates** of an effect and their uncertainty, without doing a hypothesis test.

Let us apply this four-step procedure to our example of the sum contrast.

The first step, writing down the estimated comparisons, is shown above.

The second step involves writing down the weights that each comparison gives to condition means. The weights for the first comparison are $w_{H01} = c(+2/3, -1/3, -1/3)$, and the weights for the second comparison are $w_{H02} = c(-1/3, +2/3, -1/3)$.

Before writing these into a hypothesis matrix, we also define the estimated quantity for the intercept term. The intercept parameter estimates the mean across all conditions:

$$\alpha = \frac{\mu_1 + \mu_2 + \mu_3}{3} \quad (30)$$

$$\alpha = \frac{1}{3}\mu_1 + \frac{1}{3}\mu_2 + \frac{1}{3}\mu_3 \quad (31)$$

This estimate has weights of 1/3 for all condition means. The weights from all three model parameters that were defined are now combined and written into a matrix that we refer to as the *hypothesis matrix* (H_c):

```
HcSum <- rbind(
  cH00 = c(adjectives = 1 / 3, nouns = 1 / 3, verbs = 1 / 3),
  cH01 = c(adjectives = +2 / 3, nouns = -1 / 3, verbs = -1 / 3),
  cH02 = c(adjectives = -1 / 3, nouns = +2 / 3, verbs = -1 / 3)
```

```
)
fractions(t(HcSum))

##           cH00 cH01 cH02
## adjectives  1/3  2/3 -1/3
## nouns       1/3 -1/3  2/3
## verbs       1/3 -1/3 -1/3
```

Each set of weights is first entered as a row into the matrix (command `rbind()`).² We switch rows and columns of the matrix for easier readability using the command `t()` (this transposes the matrix). The command `fractions()` from MASS library turns the decimals into fractions to improve readability.

² The reason for this is that mathematically, individual comparisons in the hypothesis matrix are coded as rows rather than as columns.

Now that the condition weights have been written into the hypothesis matrix, the third step of the procedure is implemented: the *generalized matrix inverse* is used to obtain the contrast matrix that is needed to estimate these comparisons in a linear model.

Use the function `ginv()` from the MASS package for this next step. Define a function `ginv2()` for nicer formatting of the output.^[2b]

```
# define a function to make the output nicer:
ginv2 <- function(x) {
  fractions(provideDimnames(ginv(x),
    base = dimnames(x)[2:1]
  ))
}
```

The function `fractions()` from the MASS package is used to make the output more easily readable, and the function `provideDimnames()` is used to keep row and column names.

Applying the generalized inverse to the hypothesis matrix results in the new matrix `XcSum`. This is the contrast matrix X_c that estimates exactly those comparisons that were specified earlier:

```
(XcSum <- ginv2(HcSum))
```

```
##           cH00 cH01 cH02
## adjectives  1    1    0
## nouns       1    0    1
## verbs       1   -1   -1
```

This contrast matrix corresponds exactly to the sum contrasts described above. In the case of the sum contrast, the contrast matrix looks very different from the hypothesis matrix.

The contrast matrix in sum contrasts codes with +1 the condition that is to be compared to the grand mean. The condition that is never compared to the grand mean is coded as -1 .

Without knowing the relationship between the hypothesis matrix and the contrast matrix, the meaning of the coefficients is completely opaque.

To verify this custom-made contrast matrix, it is compared to the sum contrast matrix as generated by the R function `contr.sum()` in the `stats` package. The resulting contrast matrix is identical to the result when adding the intercept term, a column of ones, to the contrast matrix:

```
fractions(cbind(1, contr.sum(3)))
```

```
##    [,1] [,2] [,3]
## 1  1    1    0
## 2  1    0    1
## 3  1   -1   -1
```

In order to estimate model parameters, step four in our procedure involves assigning sum contrasts to the factor F in our example data, and fitting a (Bayesian) linear model. This allows us to estimate the regression coefficients associated with each contrast. We compare these to the data shown above to test whether the regression coefficients actually correspond to the differences of condition means, as intended.

To define the contrast, it is necessary to remove the intercept term, as this is automatically added by the modeling function `brm()`.

```

contrasts(df_contrasts2$F) <- XcSum[, 2:3]
fit_Sum <- brm(DV ~ 1 + F,
  data = df_contrasts2,
  family = gaussian(),
  prior = c(
    prior(normal(500, 100), class = Intercept),
    prior(normal(0, 100), class = sigma),
    prior(normal(0, 100), class = b)
  )
)

fixef(fit_Sum)

##              Estimate Est.Error      Q2.5      Q97.5
## Intercept 450.49204   6.830702 436.88294 464.58804
## FcH01      -49.06511   9.838296 -69.04148 -29.49630
## FcH02       49.00675   9.637252  30.17709  67.81311

```

The (Bayesian) linear model regression coefficients show the grand mean response time of 450 ms in the intercept. Remember that the first regression coefficient FcH01 was designed to estimate the extent to which adjectives are responded to faster than the grand mean.

The regression coefficient FcH01 (Estimate) of -50 reflects the difference between adjectives (400 ms) and the grand mean of 450 ms. The second estimate of interest tells us the extent to which response times for nouns differ from the grand mean.

The fact that the second regression coefficient FcH02 is close to 50 indicates that response times for nouns are (500 ms) slower than the grand mean of 450 ms.

Although the nouns are estimated to have 50 ms longer reading times than the grand mean, the reading times for adjectives are 50 ms faster than the grand mean.

We have now not only derived contrasts, parameter estimates, and comparisons for the sum contrast, we have also used a powerful and highly general procedure that is used to generate contrasts for many kinds of different comparisons and experimental designs.

3.8 Generating contrasts: The *hypr* package

To work with the four-step procedure, i.e., to flexibly design contrasts to estimate specific comparisons, we have developed the R package *hypr*. This package allows the researcher to specify the desired comparisons, and based on these comparisons, it automatically generates contrast matrices that allow us to estimate these comparisons in linear models. The functions available in this package thus considerably simplify the implementation of the four-step procedure outlined above. Because *hypr* was originally written with the frequentist framework in mind, the comparisons are expressed as null hypotheses. In the Bayesian framework, these should be treated as comparisons between (bundles of) condition means. To illustrate the functionality of the *hypr* package, we will use the two comparisons that we had defined and analyzed in the previous section:

$$\beta_1 = \mu_1 - \frac{\mu_1 + \mu_2 + \mu_3}{3} = \mu_1 - GM \quad (32)$$

and

$$\beta_2 = \mu_2 - \frac{\mu_1 + \mu_2 + \mu_3}{3} = \mu_2 - GM \quad (33)$$

These estimates are effectively comparisons between condition means or between bundles of condition means. That is, μ_1 is compared to the grand mean and μ_2 is compared to the GM. These two comparisons can be directly entered into R using the `hypr()` function from the *hypr* package. To do so, we use some labels to indicate factor levels. E.g., adjectives, nouns, and verbs can represent factor levels μ_1 , μ_2 , and μ_3 . The first comparison specifies that μ_1 is compared to $\frac{\mu_1 + \mu_2 + \mu_3}{3}$. This can be written as a formula in R: `adjectives ~ (adjectives + nouns + verbs)/3`. The second comparison is that μ_2 is compared to $\frac{\mu_1 + \mu_2 + \mu_3}{3}$, which can be written as `nouns ~ (adjectives + nouns + verbs)/3`.

```
HcSum <- hypr(
  b1 = adjectives ~ (adjectives + nouns + verbs) / 3,
```

```

    b2 = nouns ~ (adjectives + nouns + verbs) / 3,
    levels = c("adjectives", "nouns", "verbs")
)
HcSum

## hypr object containing 2 null hypotheses:
## H0.b1: 0 = (2*adjectives - nouns - verbs)/3
## H0.b2: 0 = (2*nouns - adjectives - verbs)/3
##
## Call:
## hypr(b1 = ~2/3 * adjectives - 1/3 * nouns - 1/3 * verbs, b2 = ~2/3 *
##      nouns - 1/3 * adjectives - 1/3 * verbs, levels = c("adjectives",
## "nouns", "verbs"))
##
## Hypothesis matrix (transposed):
##           b1    b2
## adjectives 2/3 -1/3
## nouns      -1/3 2/3
## verbs      -1/3 -1/3
##
## Contrast matrix:
##           b1 b2
## adjectives 1  0
## nouns      0  1
## verbs     -1 -1

```

The results show that the comparisons between condition means have been re-written into a form where 0 is coded on the left side of the equation, and the condition means together with associated weights are written on the right side of the equation. This presentation makes it easy to see the weights of the condition means to code a certain comparison. The next part of the results shows the hypothesis matrix, which contains the weights from the condition means. Thus, `hypr` takes formulas coding comparisons between condition means as input, and automatically extracts the corresponding weights and encodes them into the hypothesis matrix. `hypr` moreover applies the generalized matrix inverse to obtain the contrast matrix from the hypothesis matrix. The different steps corre-

spond exactly to the steps we had carried out manually in the preceding section. `hypr` automatically performs these steps for us. We can now extract the contrast matrix by a simple function call:

```
contr.hypothesis(HcSum)

##           b1 b2
## adjectives  1  0
## nouns       0  1
## verbs      -1 -1
## attr(,"class")
## [1] "hypr_cmat" "matrix"    "array"
```

We can assign this contrast to our factor as we did before.

```
contrasts(df_contrasts2$F) <- contr.hypothesis(HcSum)
```

Now, we could again run the same model. However, since the contrast matrix is now the same as used before, the results would also be exactly the same, and we therefore skip the model fitting for brevity.

The `hypr` package can be used to create contrasts for Bayesian models, where the focus lies on estimation of contrasts that code comparisons between condition means or bundles of condition means.

Thus, the comparison that one specifies implies the estimation of a difference between condition means or bundles of condition means.

We see this in the output of the `hypr()` function (see the first section of the results); these formulate the comparison in a way that also illustrates the estimation of model parameters. That is, the comparison (expressed in the `hypr` package's syntax) $\mu_1 \sim \frac{\mu_1 + \mu_2 + \mu_3}{3}$ corresponds to a parameter estimate of $b1 = 2/3 * m1 - 1/3 * m2 - 1/3 * m3$, where $m1$ to $m3$ are the means for each of the conditions. The resulting contrasts will then allow us to estimate the specified differences between condition means or bundles of condition means.

3.9 Example 1: Four conditions, treatment contrasts

treatment contrasts:

```
contr.treatment(4)
```

```
##   2 3 4
## 1 0 0 0
## 2 1 0 0
## 3 0 1 0
## 4 0 0 1
```

```
h <- hypr()
```

```
cmat(h) <- contr.treatment(4)
```

```
## Warning in 'cmat<-'('*tmp*', value = structure(c(0, 1, 0, 0, 0, 0, 1, 0, : The
## contrast matrix you are assigning to this hypr object does not appear to have
## an intercept column. If this is intentional, you can ignore this warning or
## suppress it by explicitly calling cmat(..., add_intercept=FALSE) <- x.
```

```
h
```

```
## hypr object containing 3 null hypotheses:
```

```
## H0.2: 0 = X2
```

```
## H0.3: 0 = X3
```

```
## H0.4: 0 = X4
```

```
##
```

```
## Call:
```

```
## hypr('2' = ~X2, '3' = ~X3, '4' = ~X4, levels = c("X1", "X2",
## "X3", "X4"))
```

```
##
```

```
## Hypothesis matrix (transposed):
```

```
##   2 3 4
## X1 0 0 0
## X2 1 0 0
## X3 0 1 0
## X4 0 0 1
```

```
##
```

```
## Contrast matrix:
```

```
##   2 3 4
## X1 0 0 0
```

```
## X2 1 0 0
## X3 0 1 0
## X4 0 0 1

cmat(h, add_intercept = TRUE) <- contr.treatment(4)
h

## hypr object containing 4 null hypotheses:
## H0.Intercept: 0 = X1          (Intercept)
##           H0.2: 0 = -X1 + X2
##           H0.3: 0 = -X1 + X3
##           H0.4: 0 = -X1 + X4
##
## Call:
## hypr(Intercept = ~X1, '2' = ~-X1 + X2, '3' = ~-X1 + X3, '4' = ~-X1 +
##       X4, levels = c("X1", "X2", "X3", "X4"))
##
## Hypothesis matrix (transposed):
##   Intercept 2 3 4
## X1 1        -1 -1 -1
## X2 0         1 0 0
## X3 0         0 1 0
## X4 0         0 0 1
##
## Contrast matrix:
##   Intercept 2 3 4
## X1 1        0 0 0
## X2 1        1 0 0
## X3 1        0 1 0
## X4 1        0 0 1
```

The null hypotheses being tested:

$$H_{01} : \mu_1 = 0$$

$$H_{02} : \mu_2 = \mu_1$$

$$H_{03} : \mu_3 = \mu_1$$

$$H_{04} : \mu_4 = \mu_1$$

3.10 Example 2: Four conditions, Helmert contrasts

Helmert contrasts:

```
contr.helmert(4)
```

```
##      [,1] [,2] [,3]
## 1    -1   -1   -1
## 2     1   -1   -1
## 3     0    2   -1
## 4     0    0    3
```

```
h <- hypr()
```

```
cmat(h) <- contr.helmert(4)
```

```
## Warning in 'cmat<-'('*tmp*', value = structure(c(-1, 1, 0, 0, -1, -1, 2, : The
## contrast matrix you are assigning to this hypr object does not appear to have
## an intercept column. If this is intentional, you can ignore this warning or
## suppress it by explicitly calling cmat(..., add_intercept=FALSE) <- x.
```

```
h
```

```
## hypr object containing 3 null hypotheses:
```

```
## H0.1:  $0 = (-\mu_1 + \mu_2)/2$ 
```

```
## H0.2:  $0 = (-1/2\mu_1 - 1/2\mu_2 + \mu_3)/3$ 
```

```
## H0.3:  $0 = (-1/3\mu_1 - 1/3\mu_2 - 1/3\mu_3 + \mu_4)/4$ 
```

```
##
```

```
## Call:
```

```
## hypr(~-1/2 * mu1 + 1/2 * mu2, ~-1/6 * mu1 - 1/6 * mu2 + 1/3 *
```

```
##      mu3, ~-1/12 * mu1 - 1/12 * mu2 - 1/12 * mu3 + 1/4 * mu4,
```

```
##      levels = c("mu1", "mu2", "mu3", "mu4"))
```

```
##
```

```
## Hypothesis matrix (transposed):
```

```
##      [,1] [,2] [,3]
```

```
## mu1  -1/2  -1/6 -1/12
```

```
## mu2   1/2  -1/6 -1/12
```

```
## mu3     0   1/3 -1/12
```

```
## mu4     0    0  1/4
```

```
##
```

```
## Contrast matrix:
```

```
##      [,1] [,2] [,3]
```

```
## mu1 -1 -1 -1
## mu2 1 -1 -1
## mu3 0 2 -1
## mu4 0 0 3

cmat(h, add_intercept = TRUE) <- contr.helmert(4)
h

## hypr object containing 4 null hypotheses:
## H0.1: 0 = (mu1 + mu2 + mu3 + mu4)/4 (Intercept)
## H0.2: 0 = (-mu1 + mu2)/2
## H0.3: 0 = (-1/2*mu1 - 1/2*mu2 + mu3)/3
## H0.4: 0 = (-1/3*mu1 - 1/3*mu2 - 1/3*mu3 + mu4)/4
##
## Call:
## hypr(~1/4 * mu1 + 1/4 * mu2 + 1/4 * mu3 + 1/4 * mu4, ~-1/2 *
## mu1 + 1/2 * mu2, ~-1/6 * mu1 - 1/6 * mu2 + 1/3 * mu3, ~-1/12 *
## mu1 - 1/12 * mu2 - 1/12 * mu3 + 1/4 * mu4, levels = c("mu1",
## "mu2", "mu3", "mu4"))
##
## Hypothesis matrix (transposed):
##      [,1] [,2] [,3] [,4]
## mu1  1/4 -1/2 -1/6 -1/12
## mu2  1/4  1/2 -1/6 -1/12
## mu3  1/4    0  1/3 -1/12
## mu4  1/4    0    0  1/4
##
## Contrast matrix:
##      [,1] [,2] [,3] [,4]
## mu1  1 -1 -1 -1
## mu2  1 1 -1 -1
## mu3  1 0 2 -1
## mu4  1 0 0 3
```

The null hypotheses being tested:

$$H_{01} : 0 = (\mu_1 + \mu_2 + \mu_3 + \mu_4)/4$$

$$H_{02} : 0 = (-\mu_1 + \mu_2)/2$$

$$H_{03} : 0 = ((-1/2) * \mu_1 - (1/2) * \mu_2 + \mu_3) / 3$$

$$H_{04} : 0 = ((-1/3) * \mu_1 - (1/3) * \mu_2 - (1/3) * \mu_3 + \mu_4) / 4$$

3.11 *Example 3: Four conditions, sliding or repeated differences or successive differences contrast*

```
MASS::contr.sdif(4)
```

```
##      2-1  3-2  4-3
## 1 -0.75 -0.5 -0.25
## 2  0.25 -0.5 -0.25
## 3  0.25  0.5 -0.25
## 4  0.25  0.5  0.75
```

```
h <- hypr()
```

```
cmat(h) <- MASS::contr.sdif(4)
```

```
## Warning in 'cmat<-'('*tmp*', value = structure(c(-0.75, 0.25, 0.25, 0.25, : The
## contrast matrix you are assigning to this hypr object does not appear to have
## an intercept column. If this is intentional, you can ignore this warning or
## suppress it by explicitly calling cmat(..., add_intercept=FALSE) <- x.
```

```
h
```

```
## hypr object containing 3 null hypotheses:
```

```
## H0.2-1: 0 = -X1 + X2
```

```
## H0.3-2: 0 = -X2 + X3
```

```
## H0.4-3: 0 = -X3 + X4
```

```
##
```

```
## Call:
```

```
## hypr('2-1' = ~-X1 + X2, '3-2' = ~-X2 + X3, '4-3' = ~-X3 + X4,
```

```
##      levels = c("X1", "X2", "X3", "X4"))
```

```
##
```

```
## Hypothesis matrix (transposed):
```

```
##      2-1 3-2 4-3
```

```
## X1 -1   0   0
```

```
## X2  1  -1   0
```

```
## X3  0   1  -1
```

```

## X4  0   0   1
##
## Contrast matrix:
##    2-1  3-2  4-3
## X1 -3/4 -1/2 -1/4
## X2  1/4 -1/2 -1/4
## X3  1/4  1/2 -1/4
## X4  1/4  1/2  3/4

cmat(h, add_intercept = TRUE) <- MASS::contr.sdif(4)
h

## hypr object containing 4 null hypotheses:
## H0.Intercept: 0 = (X1 + X2 + X3 + X4)/4 (Intercept)
##      H0.2-1: 0 = -X1 + X2
##      H0.3-2: 0 = -X2 + X3
##      H0.4-3: 0 = -X3 + X4
##
## Call:
## hypr(Intercept = ~1/4 * X1 + 1/4 * X2 + 1/4 * X3 + 1/4 * X4,
##      '2-1' = ~-X1 + X2, '3-2' = ~-X2 + X3, '4-3' = ~-X3 + X4,
##      levels = c("X1", "X2", "X3", "X4"))
##
## Hypothesis matrix (transposed):
##   Intercept 2-1 3-2 4-3
## X1 1/4      -1   0   0
## X2 1/4       1  -1   0
## X3 1/4       0   1  -1
## X4 1/4       0   0   1
##
## Contrast matrix:
##   Intercept 2-1 3-2 4-3
## X1    1     -3/4 -1/2 -1/4
## X2    1      1/4 -1/2 -1/4
## X3    1      1/4  1/2 -1/4
## X4    1      1/4  1/2  3/4

```

The null hypotheses are printed out by hypr above.

4 Contrast coding for multifactorial designs

4.1 ANOVA-style coding

Above, we used a data set with one 4-level factor. Here, we assume that the same four means come from an $A(2) \times B(2)$ between-subject-factor design rather than an $F(4)$ between-subject-factor design.

Load the simulated data and show summary statistics as a table and in Figure 3. The means and standard deviations are exactly the same as in the previous example.

```
## Ignoring unknown labels:
## * colour : "Factor B"
```

```
table4a
```

```
##   Factor A Factor B N data Means Std. dev. Std. errors
## 1      A1      B1    5    10         10    4.472136
## 2      A1      B2    5    20         10    4.472136
## 3      A2      B1    5    10         10    4.472136
## 4      A2      B2    5    40         10    4.472136
```

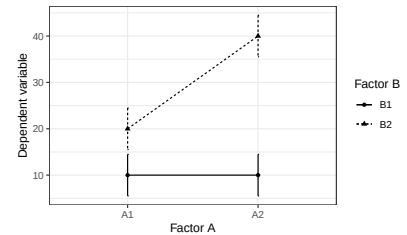


Figure 3: Means and error bars (showing standard errors) for a simulated data set with a two-by-two between-subjects factorial design.

A standard ANOVA analysis:

```
omnibus_anova<-anova_test(data = df_contrasts4,
                           dv = DV,
                           between = c(A,B)
                           )
```

```
get_anova_table(omnibus_anova)
```

```
## ANOVA Table (type II tests)
```

```
##
```

```
##   Effect DFn DFd  F      p p<.05    ges
## 1      A   1  16   5 0.040000    * 0.238
## 2      B   1  16 20 0.000385    * 0.556
## 3     A:B   1  16   5 0.040000    * 0.238
```

In order to carry out a 2×2 ANOVA-type (main effects and interaction) analysis, one needs sum contrasts in the

linear model. (This is true for factors with two levels, but does not generalize to factors with more than two levels.)

The results of such an analysis are shown below. (This analysis uses sum contrasts: $-1/+1$; however, scaled sum contrasts $-0.5/+0.5$ could also be used; in this case, the priors would need to be adapted to the respective scale.)

```
# define sum contrasts:
contrasts(df_contrasts4$A) <- contr.sum(2)
contrasts(df_contrasts4$B) <- contr.sum(2)
```

Here is the frequentist analysis:

```
## Frequentist LM:
fit_AB.sum_lm<-lm(DV ~ 1 + A * B,
  data = df_contrasts4)
summary(fit_AB.sum_lm)

##
## Call:
## lm(formula = DV ~ 1 + A * B, data = df_contrasts4)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -16.6833  -6.3020   0.3401   7.6611  14.7131
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    20.000      2.236   8.944 1.27e-07 ***
## A1             -5.000      2.236  -2.236 0.039945 *
## B1            -10.000      2.236  -4.472 0.000385 ***
## A1:B1           5.000      2.236   2.236 0.039945 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 10 on 16 degrees of freedom
## Multiple R-squared:  0.6522, Adjusted R-squared:  0.587
## F-statistic:    10 on 3 and 16 DF,  p-value: 0.0005943

t_values<-summary(fit_AB.sum_lm)$coefficients[2:4,3]
## t^2 = F scores from an ANOVA:
t_values^2
```

```
##      A1      B1 A1:B1
##      5      20      5
```

The Bayesian model:

```
# Bayesian LM
fit_AB.sum <- brm(DV ~ 1 + A * B,
  data = df_contrasts4,
  family = gaussian(),
  prior = c(
    prior(normal(20, 50), class = Intercept),
    prior(normal(0, 50), class = sigma),
    prior(normal(0, 50), class = b)
  )
)
```

tab18

```
##      Predictor Estimate Est. Error 2.5\\% 97.5\\%
## 1 Intercept      20.00      2.46  15.13  24.91
## 2          A1      -4.95      2.45  -9.69  -0.13
## 3          B1     -9.96      2.39 -14.56  -5.24
## 4       A1:B1       5.00      2.49   0.09   9.96
```

Next, we reproduce the $A(2) \times B(2)$ - ANOVA with contrasts specified for the corresponding one-way $F(4)$ ANOVA, that is by treating the $2 \times 2 = 4$ condition means as four levels of a single factor F . In other words, we go back to the data frame simulated for the analysis of repeated/sliding contrasts.

We first define weights for condition means according to our hypotheses, invert this matrix, and use it as the contrast matrix for factor F . We define weights of $1/4$ and $-1/4$.

We do so because (a) we want to compare the mean of two conditions to the mean of two other conditions (e.g., factor A compares $\frac{F1+F2}{2}$ to $\frac{F3+F4}{2}$).

Moreover, (b) we want coefficients to code half the difference between condition means; this scaling will lead to sum contrasts as above once we invert the hypothesis matrix.

```

t(fractions(HcInt <- rbind(
  A = c(F1 = 1 / 4, F2 = 1 / 4, F3 = -1 / 4, F4 = -1 / 4),
  B = c(F1 = 1 / 4, F2 = -1 / 4, F3 = 1 / 4, F4 = -1 / 4),
  AxB = c(F1 = 1 / 4, F2 = -1 / 4, F3 = -1 / 4, F4 = 1 / 4)
)))

##      A      B      AxB
## F1  1/4   1/4   1/4
## F2  1/4  -1/4  -1/4
## F3 -1/4   1/4  -1/4
## F4 -1/4  -1/4   1/4

(XcInt <- ginv2(HcInt))

##      A  B  AxB
## F1   1  1  1
## F2   1 -1 -1
## F3  -1  1 -1
## F4  -1 -1  1

contrasts(df_contrasts3$F) <- XcInt

```

Again, I fit the frequentist and Bayesian models:

```

fit_F4.sum_lm <- lm(DV ~ 1 + F, data = df_contrasts3)
summary(fit_F4.sum_lm)

##
## Call:
## lm(formula = DV ~ 1 + F, data = df_contrasts3)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -16.6833  -6.3020   0.3401   7.6611  14.7131
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    20.000      2.236   8.944 1.27e-07 ***
## FA             -5.000      2.236  -2.236 0.039945 *
## FB            -10.000      2.236  -4.472 0.000385 ***
## FAxB             5.000      2.236   2.236 0.039945 *
## ---

```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 10 on 16 degrees of freedom
## Multiple R-squared:  0.6522, Adjusted R-squared:  0.587
## F-statistic:    10 on 3 and 16 DF,  p-value: 0.0005943

fit_F4.sum <- brm(DV ~ 1 + F,
  data = df_contrasts3,
  family = gaussian(),
  prior = c(
    prior(normal(20, 50), class = Intercept),
    prior(normal(0, 50), class = sigma),
    prior(normal(0, 50), class = b)
  )
)

fixef(fit_F4.sum)
```

##	Estimate	Est.Error	Q2.5	Q97.5
## Intercept	20.023885	2.524878	15.1511148	25.1734127
## FA	-4.988349	2.448349	-9.9167982	-0.2972884
## FB	-10.030291	2.424851	-14.7777293	-5.3927276
## FAxB	4.960224	2.413792	0.2230882	9.8506317

This shows that it is possible to specify the contrasts not only for each factor (e.g., here in the 2×2 design) separately.

Instead, one can also pool all experimental conditions (or design cells) into one large factor (here factor F with 4 levels), and specify the contrasts for the main effects and for the interactions in the resulting one large contrast matrix simultaneously.

In this approach, it can again be very useful to apply the `hypr` package to construct contrasts for a 2×2 design.

- The first parameter estimates the main effect A , i.e., it compares the average of $F1$ and $F2$ to the average of $F3$ and $F4$.
- The second parameter estimates the main effect B , i.e., it compares the average of $F1$ and $F3$ to the average of $F2$ and $F4$.

We code direct differences between the averages, i.e., we implement scaled sum contrasts instead of sum contrasts. This is shown below: the contrast matrix contains coefficients of $+1/2$ and $-1/2$ instead of $+1$ and -1 . The interaction term estimates the difference between differences, i.e., the difference between $F1 - F2$ and $F3 - F4$.

```
hAxB <- hypr(
  A = (F1 + F2) / 2 ~ (F3 + F4) / 2,
  B = (F1 + F3) / 2 ~ (F2 + F4) / 2,
  AxB = (F1 - F2) ~ (F3 - F4)
)
hAxB

## hypr object containing 3 null hypotheses:
##   H0.A: 0 = (F1 + F2 - F3 - F4)/2
##   H0.B: 0 = (F1 + F3 - F2 - F4)/2
## H0.AxB: 0 = F1 - F2 - F3 + F4
##
## Call:
## hypr(A = ~1/2 * F1 + 1/2 * F2 - 1/2 * F3 - 1/2 * F4, B = ~1/2 *
##   F1 + 1/2 * F3 - 1/2 * F2 - 1/2 * F4, AxB = ~F1 - F2 - F3 +
##   F4, levels = c("F1", "F2", "F3", "F4"))
##
## Hypothesis matrix (transposed):
##   A    B    AxB
## F1 1/2 1/2    1
## F2 1/2 -1/2   -1
## F3 -1/2 1/2   -1
## F4 -1/2 -1/2    1
##
## Contrast matrix:
##   A    B    AxB
## F1 1/2 1/2 1/4
## F2 1/2 -1/2 -1/4
## F3 -1/2 1/2 -1/4
## F4 -1/2 -1/2 1/4

contrasts(df_contrasts3$F) <- contr.hypothesis(hAxB)
```

The linear model output:

```

fit_F4hypr_lm <- lm(DV ~ 1 + F,
  data = df_contrasts3)
summary(fit_F4hypr_lm)

##
## Call:
## lm(formula = DV ~ 1 + F, data = df_contrasts3)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -16.6833  -6.3020   0.3401   7.6611  14.7131
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    20.000      2.236   8.944 1.27e-07 ***
## FA             -10.000      4.472  -2.236 0.039945 *
## FB             -20.000      4.472  -4.472 0.000385 ***
## FAxB           20.000      8.944   2.236 0.039945 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 10 on 16 degrees of freedom
## Multiple R-squared:  0.6522, Adjusted R-squared:  0.587
## F-statistic:    10 on 3 and 16 DF,  p-value: 0.0005943

## compare with estimates in the earlier model with +/- 1 coding:
summary(fit_AB.sum_lm)

##
## Call:
## lm(formula = DV ~ 1 + A * B, data = df_contrasts4)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -16.6833  -6.3020   0.3401   7.6611  14.7131
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    20.000      2.236   8.944 1.27e-07 ***
## A1             -5.000      2.236  -2.236 0.039945 *

```

```
## B1          -10.000      2.236  -4.472 0.000385 ***
## A1:B1        5.000      2.236   2.236 0.039945 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 10 on 16 degrees of freedom
## Multiple R-squared:  0.6522, Adjusted R-squared:  0.587
## F-statistic:    10 on 3 and 16 DF,  p-value: 0.0005943

fit_F4hypr <- brm(DV ~ 1 + F,
  data = df_contrasts3,
  family = gaussian(),
  prior = c(
    prior(normal(20, 50), class = Intercept),
    prior(normal(0, 50), class = sigma),
    prior(normal(0, 50), class = b)
  )
)

fixef(fit_F4hypr)

##           Estimate Est.Error      Q2.5      Q97.5
## Intercept  20.004582  2.448170  15.2946504  24.80856764
## FA        -9.806585  4.990966 -19.6109193 -0.02574264
## FB       -19.750675  5.103911 -29.4155720 -9.53494036
## FAxB       19.156760  9.612647   0.1020566  38.39777715
```

The results show that the estimates for the main effects have double the size as compared to the sum contrasts—this is the result of the scaling that we applied.

I.e., **the main effects now directly estimate the difference between averages**. The interaction estimates exactly the **difference between differences**.

In my view, the best approach is to use $\pm\frac{1}{2}$ coding for the main effects and then derive the interaction from these, because this allows us to directly estimate the differences of interest, and **because the prior specification can be easily worked out using prior knowledge and/or beliefs**.

Importantly, if we change the scaling of the contrasts, we also need to adapt the scaling of the priors. If ade-

quate priors are used, then both contrasts would lead to the same hypothesis tests if one were doing hypothesis testing using Bayes factors.

4.2 Practical relevance of scaling for prior specification for interactions

In a recent replication attempt³ of a published paper,⁴ while analyzing a 2×2 design, we used as a prior for the interaction term, which was of primary interest scientifically.

The experiment design had four conditions:

Non-locally coherent conditions

1. **Unreduced Relative Clause:** The coach smiled at the player who was *thrown* a frisbee by the opposing team.
2. **Reduced Relative Clause:** The coach smiled at the player *thrown* a frisbee by the opposing team.

Locally coherent conditions

3. **Unreduced Relative Clause:** The coach smiled at the player who was *tossed* a frisbee by the opposing team.
4. **Reduced Relative Clause:** The coach smiled at *the player tossed a frisbee* by the opposing team.

The theoretical claim is that only in condition 4 will the reader momentarily be misled by the phrase *the player tossed a frisbee* as an active sentence. This is expected to lead to a slowdown either at the verb tossed in condition 4, or at a region beyond the word. None of the other conditions should show such a momentary mis-parse. This is a famous phenomenon called local coherence, and it is theoretically very important because it implies that the parser ignores crucial grammatical information: in 4, the sentence is clearly a reduced relative, so it cannot be the case that the player was the one tossing the frisbee.

The original authors (Tabor et al. 2004) found that the interaction between the reduction of the relative clause and the ambiguity of the participle was superadditive, with a sizable interaction estimate of 73,ms (95% CI:

³ Dario Paape, Garrett Smith, and Shravan Vasishth. Do local coherence effects exist in English reduced relative clauses? *Journal of Memory and Language*, 140, 2025.

⁴ Tabor, W., Galantucci, B., & Richardson, D. (2004). Effects of merely local syntactic coherence on sentence processing. *Journal of Memory and Language*, 50(4), 355-370.

[16,ms, 132,ms]). This estimate was computed from Table 1 in the Tabor et al paper: The by-subjects F-score was 6.32, and the effect size of the interaction is reported there to be 73 ms. Because the F-score is the square of the t-value, we can compute the standard error by solving for SE in $\sqrt{6.32} = \frac{73-0}{SE}$, and then computing $73 \pm 2 \times SE$ to obtain an approximation of the 95% confidence interval for the interaction. With this information, we can now define an informative prior for the interaction; this informative directional prior is needed for a replication Bayes factor analysis [see Ly et al 2018].

Here, it is important to code the interaction such that the Bayesian model yields exactly posterior of the difference of differences. If we had coded the terms as ± 1 , we would be using the wrong prior based on the original study!

4.3 Using *ifelse* to code contrasts

An alternative way to code main effects and interactions is to use the *ifelse* command in R. For example, if we want to use ± 1 sum contrasts in the above example, we can specify the contrasts for the main effects as vectors:

```
A <- ifelse(df_contrasts3$F %in% c("F1", "F2"), -1, 1)
B <- ifelse(df_contrasts3$F %in% c("F1", "F3"), -1, 1)
```

Now, defining the interaction is simply a matter of multiplying the two vectors:

```
AxB <- A * B
```

Of course, we can use $\pm 1/2$ coding when using this approach:

```
A <- ifelse(df_contrasts3$F %in% c("F1", "F2"), -1 / 2, 1 / 2)
B <- ifelse(df_contrasts3$F %in% c("F1", "F3"), -1 / 2, 1 / 2)
## interaction:
(AxB <- A * B)

## [1]  0.25  0.25  0.25  0.25  0.25  0.25 -0.25 -0.25 -0.25 -0.25 -0.25 -0.25 -0.25
## [13] -0.25 -0.25 -0.25  0.25  0.25  0.25  0.25  0.25
```

As mentioned above, the $\pm \frac{1}{2}$ approach is better because the main effects and interaction can be directly interpreted as differences between averages and as differences between differences.

This kind of vector-based contrast coding is convenient for more complex designs, such as $2 \times 2 \times 2$ factorial designs.⁵

⁵ Example: Lena A. Jäger, Daniela Mertzen, Julie A. Van Dyke, and Shrawan Vasishth. Interference patterns in subject-verb agreement and reflexives revisited: A large-sample study. *Journal of Memory and Language*, 111, 2020.

4.4 Using merge to code contrasts

A neater way to set up contrasts “by hand” is to use the merge function.

Suppose we have a 2x2 factorial design with the factor levels specified as shown below:

```
data(df_persianE1)
head(df_persianE1)

##      subj item   rt distance  predability
## 60      4    6  568    short predictable
## 94      4   17  517     long unpredictable
## 146     4   22  675    short predictable
## 185     4    5  575     long unpredictable
## 215     4    3  581     long predictable
## 285     4    7 1171     long predictable

distance<-rep(c("short","long"),each=2)
predability<-rep(c("predictable","unpredictable"),2)
ME_dist<-rep(c(1/2,-1/2),each=2)
ME_pred<-rep(c(1/2,-1/2),2)
contrast_coding<-data.frame(distance,predability,ME_dist,ME_pred)

dat<-merge(df_persianE1,contrast_coding,
           by.x=c("distance","predability"),
           by.y=c("distance","predability"))

dim(df_persianE1)

## [1] 1512    5

dim(dat)
```

```
## [1] 1512    7
```

```
head(dat)
```

```
## distance predictability subj item rt ME_dist ME_pred
## 1      long predictable   19   4  516    -0.5    0.5
## 2      long predictable    5  18  558    -0.5    0.5
## 3      long predictable   24  35 1121    -0.5    0.5
## 4      long predictable   38  29  488    -0.5    0.5
## 5      long predictable   27  28  456    -0.5    0.5
## 6      long predictable   19  12  332    -0.5    0.5
```

```
dat$Int<-dat$ME_dist*dat$ME_pred
```

```
head(dat)
```

```
## distance predictability subj item rt ME_dist ME_pred Int
## 1      long predictable   19   4  516    -0.5    0.5 -0.25
## 2      long predictable    5  18  558    -0.5    0.5 -0.25
## 3      long predictable   24  35 1121    -0.5    0.5 -0.25
## 4      long predictable   38  29  488    -0.5    0.5 -0.25
## 5      long predictable   27  28  456    -0.5    0.5 -0.25
## 6      long predictable   19  12  332    -0.5    0.5 -0.25
```

4.5 Nested effects

One can estimate effects that do not correspond directly to main effects and interaction of the traditional ANOVA.

For example, in a 2×2 experimental design, where factor A codes word frequency (low/high) and factor B is part of speech (noun/verb), one can estimate the effect of word frequency within nouns and the effect of word frequency within verbs.

Formally, A_{B1} versus A_{B2} are nested within levels of B . Said differently, simple effects of factor A are estimated for each of the levels of factor B .

In this version, we estimate the main effect of part of speech (B ; as in traditional ANOVA). Instead of also estimating the second main effect word frequency, A , and the interaction, we estimate whether

- (1) the difference in the two levels of word frequency A in the first level of B (i.e., nouns), and

- (2) the difference in the two levels of word frequency, A in the second level of B (i.e., verbs). Often, researchers have hypotheses about these differences, and not about the interaction.

```
t(fractions(HcNes <- rbind(
  B = c(F1 = 1 / 2, F2 = -1 / 2, F3 = 1 / 2, F4 = -1 / 2),
  B1xA = c(F1 = -1, F2 = 0, F3 = 1, F4 = 0),
  B2xA = c(F1 = 0, F2 = -1, F3 = 0, F4 = 1)
)))
```

```
##      B      B1xA B2xA
## F1  1/2    -1     0
## F2 -1/2     0    -1
## F3  1/2     1     0
## F4 -1/2     0     1
```

```
(XcNes <- ginv2(HcNes))
```

```
##      B      B1xA B2xA
## F1  1/2 -1/2     0
## F2 -1/2     0 -1/2
## F3  1/2  1/2     0
## F4 -1/2     0  1/2
```

```
contrasts(df_contrasts3$F) <- XcNes
```

```
fit_Nest <- brm(DV ~ 1 + F,
  data = df_contrasts3,
  family = gaussian(),
  prior = c(
    prior(normal(20, 50), class = Intercept),
    prior(normal(0, 50), class = sigma),
    prior(normal(0, 50), class = b)
  )
)
```

```
fixef(fit_Nest)
```

```
##              Estimate Est.Error      Q2.5      Q97.5
## Intercept  19.9957902   2.420183  15.251438  24.91061
## FB        -19.9527991   4.870301 -30.184924 -10.82281
```

```
## FB1xA      0.1268126  7.044190 -13.196276  13.70695
## FB2xA      19.5139259  7.026726   5.509905  33.29246
```

The regression coefficients estimate the grand mean, the difference for the main effect of part of speech (B) and the two differences (for A ; i.e., simple main effects) within the two levels (noun and verb) of part of speech (B).

These custom nested contrasts' columns are scaled versions of the corresponding hypothesis matrix. This is the case because the columns are orthogonal. It illustrates one advantage of orthogonal contrasts for the interpretation of regression coefficients: the underlying comparisons being estimated are already clear from the contrast matrix.

In cases such as these, where A_{B1} vs. A_{B2} are nested within levels of B , it is necessary to include the effect of B (part of speech) in the model, even if one is only interested in the effect of A (word frequency) within levels of B (part of speech).

Leaving out factor B in this case would increase posterior uncertainty in the case of fully balanced data, and can lead to biases in parameter estimation in the case the data are not fully balanced.

Again, we show how nested contrasts can be easily implemented using `hypr`:

```
hNest <- hypr(
  B = (F1 + F3) / 2 ~ (F2 + F4) / 2,
  B1xA = F3 ~ F1,
  B2xA = F4 ~ F2
)
hNest

## hypr object containing 3 null hypotheses:
## H0.B: 0 = (F1 + F3 - F2 - F4)/2
## H0.B1xA: 0 = F3 - F1
## H0.B2xA: 0 = F4 - F2
##
## Call:
## hypr(B = ~1/2 * F1 + 1/2 * F3 - 1/2 * F2 - 1/2 * F4, B1xA = ~F3 -
## F1, B2xA = ~F4 - F2, levels = c("F1", "F2", "F3", "F4"))
```

```
##
## Hypothesis matrix (transposed):
##      B      B1xA B2xA
## F1  1/2    -1     0
## F2 -1/2     0    -1
## F3  1/2     1     0
## F4 -1/2     0     1
##
## Contrast matrix:
##      B      B1xA B2xA
## F1  1/2 -1/2     0
## F2 -1/2     0 -1/2
## F3  1/2  1/2     0
## F4 -1/2     0  1/2

contrasts(df_contrasts3$F) <- contr.hypothesis(hNest)

fit_NestHypr <- brm(DV ~ 1 + F,
  data = df_contrasts3,
  family = gaussian(),
  prior = c(
    prior(normal(20, 50), class = Intercept),
    prior(normal(0, 50), class = sigma),
    prior(normal(0, 50), class = b)
  )
)

fixef(fit_NestHypr)
```

##		Estimate	Est.Error	Q2.5	Q97.5
##	Intercept	19.91053370	2.512716	14.951256	24.78937
##	FB	-19.78585418	4.927283	-29.504965	-10.13479
##	FB1xA	-0.02022505	7.267631	-14.264171	14.08113
##	FB2xA	19.39786467	6.955096	5.398278	33.20799

Of course, we can also ask the reverse question: Are there differences for part of speech (B) in the levels of word frequency (A ; in addition to estimating the main effect of word frequency, A)? That is, do nouns differ from verbs for low-frequency words (B_{A1}) and do nouns differ from verbs for high-frequency words (B_{A2})?

```

hNest2 <- hypr(
  A = (F1 + F2) / 2 ~ (F3 + F4) / 2,
  A1xB = F2 ~ F1,
  A2xB = F4 ~ F3
)
hNest2

## hypr object containing 3 null hypotheses:
##   H0.A: 0 = (F1 + F2 - F3 - F4)/2
## H0.A1xB: 0 = F2 - F1
## H0.A2xB: 0 = F4 - F3
##
## Call:
## hypr(A = ~1/2 * F1 + 1/2 * F2 - 1/2 * F3 - 1/2 * F4, A1xB = ~F2 -
##   F1, A2xB = ~F4 - F3, levels = c("F1", "F2", "F3", "F4"))
##
## Hypothesis matrix (transposed):
##   A    A1xB A2xB
## F1  1/2   -1    0
## F2  1/2    1    0
## F3 -1/2    0   -1
## F4 -1/2    0    1
##
## Contrast matrix:
##   A    A1xB A2xB
## F1  1/2 -1/2    0
## F2  1/2  1/2    0
## F3 -1/2    0 -1/2
## F4 -1/2    0  1/2

contrasts(df_contrasts3$F) <- contr.hypothesis(hNest2)

fit_Nest2Hypr <- brm(DV ~ 1 + F,
  data = df_contrasts3,
  family = gaussian(),
  prior = c(
    prior(normal(20, 50), class = Intercept),
    prior(normal(0, 50), class = sigma),
    prior(normal(0, 50), class = b)
  )
)

```

)

```
fixef(fit_Nest2Hypr)
```

##	Estimate	Est.Error	Q2.5	Q97.5
## Intercept	20.000735	2.460581	15.288435	24.9951485
## FA	-10.015680	4.823360	-19.680749	-0.6625486
## FA1xB	9.576277	7.072347	-4.427062	23.7799234
## FA2xB	29.308664	6.847409	15.257203	42.7967363

Regression coefficients estimate the grand mean, the difference for the main effect of word frequency (A) and the two part of speech effects (for B ; i.e., simple main effects) within levels of word frequency (A).